



ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

School of Electrical Engineering and Computing

Department of Computer Science and Engineering

Course: Web Programming

Title: Hospital Management System

Group members	ID no.
1. Bekam Genene	UGR/30253/15
2. Begonet Debebe	UGR/30244/15
3. Endrias Eshetu	UGR/30469/15
4. Kenenisa Beyan	UGR/30772/15
5. Yeabsira Goitom	UGR/31390/15

Submitted to : Mr. Merga Woldeyohannis
Submission date: May 22, 2026

Table of Contents

1. Executive Summary	3
2. Project Objectives	3
3. Five-Day Development Delivery Plan.....	4
4. Technology Stack.....	5
5. Functional Requirements	6
5.1 Authentication	6
5.2 User Roles	6
5.3 Patient Features	6
5.4 Doctor Features	7
5.5 Administrator Features	7
6. Appointment Management.....	8
6.1 Supported Operations	8
6.2 Appointment Status Workflow.....	8
7. Medical Records Management	9
7.1 Record Structure.....	9
7.2 Access Control	9
8. Prescription Management	9
8.1 Prescription Structure.....	9
8.2 Access Control	10
9. Notification System	10
9.1 Notification Events.....	10
9.2 Technical Implementation.....	10
10. Database Models.....	10
11. API Endpoint Reference	11
12. User Interface — Page Structure	13
13. Non-Functional Requirements	14
14. Planned Future Enhancements.....	15
15. Conclusion	16

1. Executive Summary

This document constitutes the formal Technical Requirements Document (TRD) for the Hospital Management System (HMS) Minimum Viable Product (MVP). The HMS is a full-stack healthcare web application engineered to digitize, streamline, and automate core hospital workflows, including patient registration, appointment scheduling, medical record management, and digital prescription issuance.

The system is architected to serve three distinct user roles Patient, Doctor, and Administrator each with a tailored interface and defined access privileges enforced through role-based access control (RBAC). The MVP is designed as a foundation for future expansion into broader hospital-scale modules including billing, laboratory management, and admission tracking.

2. Project Objectives

The Hospital Management System MVP is designed to achieve the following strategic objectives:

- ✓ Digitize end-to-end hospital appointment workflows, eliminating dependency on manual, paper-based processes.
- ✓ Reduce administrative overhead associated with manual patient registration and record keeping.
- ✓ Improve physician schedule management and reduce scheduling conflicts through automated availability tracking.
- ✓ Provide a secure, centralized repository for patient medical records, accessible by authorized personnel only.
- ✓ Enable physicians to issue, manage, and communicate prescriptions digitally.
- ✓ Establish a scalable, modular architecture that supports future expansion into additional hospital service domains.

3. Five-Day Development Delivery Plan

The following schedule outlines the structured development plan for delivering the HMS MVP within a five-day timeframe. Each day is allocated to specific modules to ensure focused, sequential, and traceable progress.

Day	Phase	Deliverables	Key Milestones
Day 1	Foundation & Authentication	Project scaffolding (Next.js + Express), MongoDB connection, User model, JWT authentication, bcrypt password hashing, refresh token logic, HTTP-only cookies, Auth API endpoints (register / login / logout).	Secure auth flow operational; role-based middleware active.
Day 2	Patient & Doctor Modules	Patient model & profile management, Doctor model & availability configuration, CRUD API endpoints for patients and doctors, Admin management interfaces for both entities.	Patient registration, profile updates, and doctor onboarding functional.
Day 3	Appointment Management	Appointment model with status workflow (Pending, Confirmed, Completed, Cancelled), Booking, cancellation, and rescheduling APIs, Conflict detection and double-booking prevention, Availability management logic.	Full appointment lifecycle management operational.
Day 4	Medical Records & Prescriptions	Medical Record model (symptoms, diagnosis, clinical notes), Prescription model with medicine line items (name, dosage, duration), Record creation and retrieval APIs, Role-specific access enforcement for records.	Doctors can create records and prescriptions; patients can view their own.
Day 5	Frontend, Notifications & QA	React/Next.js UI pages (Admin, Doctor, Patient dashboards), BullMQ + Node-cron background job setup, Resend email	Fully integrated, tested MVP ready for deployment.

		integration (reminders, confirmations), End-to-end testing, bug fixes, and deployment preparation.	
--	--	--	--

4. Technology Stack

The following technologies have been selected based on their suitability for rapid, scalable full-stack development within the healthcare domain.

Layer	Technology	Purpose
Frontend	React.js / Next.js	Server-side rendered, SEO-capable frontend framework.
Styling	Tailwind CSS	Utility-first CSS framework for responsive, custom UI design.
State Management	Zustand	Lightweight global client-side state management.
Server State	TanStack Query	Asynchronous server state management with caching and synchronization.
HTTP Client	Axios	Promise-based HTTP client for API communication.
Backend	Express.js	Minimal, performant Node.js web application framework.
Database	MongoDB + Mongoose	NoSQL document database with object-document mapping (ODM).
Job Scheduling	BullMQ + Node-cron	Redis-backed message queue and cron-based scheduled task runner.
Email Service	Resend	Developer-oriented transactional email delivery service.
Authentication	JWT + bcrypt	Stateless token-based authentication with secure password hashing.

Note: No third-party component libraries are to be utilized. All UI components shall be constructed manually using React.js and Tailwind CSS to ensure full design control and avoid external dependencies.

5. Functional Requirements

5.1 Authentication

All users must authenticate via a secure login flow. The system shall support the following authentication operations:

- ✓ User account registration via email and password.
- ✓ Secure login using email and password with JWT issuance.
- ✓ Secure logout with token invalidation via HTTP-only cookies.
- ✓ Refresh token mechanism for session continuity.

5.2 User Roles

The system enforces three distinct user roles, each with a defined set of permissions and interface access:

Role	Description
Patient	End users who register, book appointments, and access their health records.
Doctor	Healthcare professionals who manage consultations, create records, and issue prescriptions.
Admin	Platform administrators who oversee all users, appointments, and system-wide operations.

5.3 Patient Features

Authenticated patients shall be permitted to perform the following operations within the system:

- ✓ Register a patient profile linked to their user account.
- ✓ Update personal profile information (age, gender, blood group, contact details, address).
- ✓ Book new appointments with available doctors.
- ✓ Cancel existing appointments subject to applicable business rules.
- ✓ Reschedule confirmed appointments to alternative available time slots.

- ✓ View complete appointment history with status tracking.
- ✓ View prescriptions issued by treating physicians.
- ✓ View associated medical records created during consultations.

5.4 Doctor Features

Authenticated doctors shall be permitted to perform the following operations within the system:

- ✓ Log in securely using credentials provisioned by the administrator.
- ✓ View all appointments assigned to their profile, filterable by date and status.
- ✓ Access patient demographic and contact information for scheduled consultations.
- ✓ Create and submit medical records containing clinical observations.
- ✓ Record diagnosis and associated clinical notes for patient consultations.
- ✓ Issue digital prescriptions containing one or more medicine line items.

5.5 Administrator Features

Authenticated administrators shall be permitted to perform the following operations within the system:

- ✓ Create, update, and deactivate doctor accounts and profiles.
- ✓ View and manage patient accounts and associated records.
- ✓ Monitor, update, and manage all appointments across the platform.
- ✓ Access a real-time administrative dashboard displaying the following key metrics:
 - Total registered patients.
 - Total active doctors.
 - Total appointments by status.

6. Appointment Management

The appointment management module constitutes the core operational workflow of the HMS. The system shall enforce the following appointment lifecycle and business rules.

6.1 Supported Operations

- ✓ Appointment booking by patients against available doctor time slots.
- ✓ Appointment cancellation by patients or administrators.
- ✓ Appointment rescheduling to an alternative available slot.
- ✓ Doctor availability management, including available days and time slots.
- ✓ Real-time booking status tracking throughout the appointment lifecycle.

6.2 Appointment Status Workflow

Status	Description
Pending	Appointment has been submitted by the patient and is awaiting doctor or administrator confirmation.
Confirmed	Appointment has been reviewed and confirmed. The patient is scheduled for the designated time slot.
Completed	The consultation has concluded. The doctor may now create associated medical records and prescriptions.
Cancelled	The appointment has been cancelled by the patient, doctor, or administrator prior to the scheduled time.

The system shall enforce conflict detection logic to prevent double-booking of a doctor within the same time slot. All booking requests shall be validated against existing confirmed appointments before acceptance.

7. Medical Records Management

The medical records module enables treating physicians to document clinical observations associated with a completed appointment. All records are immutably linked to a specific patient, doctor, and appointment instance.

7.1 Record Structure

Each medical record shall contain the following structured fields:

- Symptoms: A description of the patient-reported and clinically observed symptoms.
- Diagnosis: The physician's formal clinical diagnosis based on examination and symptoms.
- Clinical Notes: Additional observations, follow-up instructions, or contextual remarks made by the treating physician.

7.2 Access Control

- ✓ Doctors may create medical records exclusively for patients under their care.
- ✓ Patients may view their own medical records but may not create or modify any records.
- ✓ Administrators may view all records for operational oversight purposes.

8. Prescription Management

The prescription management module allows physicians to issue structured digital prescriptions linked to a patient encounter. Prescriptions are accessible by the issuing doctor, the receiving patient, and system administrators.

8.1 Prescription Structure

Each prescription shall contain one or more medicine line items, each including the following attributes:

- ✓ Medicine Name: The proprietary or generic name of the prescribed pharmaceutical.
- ✓ Dosage: The prescribed quantity or concentration per administration (e.g., 500 mg).

- ✓ Duration: The period over which the medication is to be administered (e.g., 7 days).

8.2 Access Control

- ✓ Doctors may issue prescriptions for any patient with whom they have a confirmed or completed appointment.
- ✓ Patients may view prescriptions issued under their profile but may not alter them.

9. Notification System

The notification system is responsible for the automated dispatch of transactional and reminder emails to system users. Notifications are processed asynchronously through a background job queue to ensure non-blocking delivery.

9.1 Notification Events

- ✓ Registration Confirmation: Dispatched to new users upon successful account creation.
- ✓ Appointment Reminders: Dispatched to patients in advance of scheduled appointments.

9.2 Technical Implementation

- ✓ BullMQ: Redis-backed job queue for reliable, asynchronous email job processing.
- ✓ Node-cron: Scheduled job runner for time-triggered tasks (e.g., daily reminder dispatch).
- ✓ Resend: Transactional email delivery service for outbound email communication.

10. Database Models

The following MongoDB document schemas define the data architecture of the HMS. All models are implemented using Mongoose ODM.

Model	Key Fields	Description
User	name, email, password, role	Base authentication record for all system users. Role determines access level.
Patient	userId, patientId, age, gender, bloodGroup, phone, address	Extended profile linked to a User record. Stores clinical and demographic attributes.

Doctor	userId, specialization, department, availableDays	Professional profile linked to a User record. Defines scheduling availability.
Appointment	patientId, doctorId, date, time, status	Represents a scheduled encounter between a patient and a doctor with lifecycle status tracking.
MedicalRecord	patientId, doctorId, appointmentId, symptoms, diagnosis, notes	Clinical documentation linked to a completed appointment. Immutable post-creation.
Prescription	patientId, doctorId, medicines[]	Digital prescription record. Each medicine entry includes name, dosage, and duration.

11. API Endpoint Reference

All API endpoints follow RESTful conventions. Authentication is enforced via JWT tokens transmitted through HTTP-only cookies. Role-based middleware restricts access at the route level.

Method	Endpoint	Description
Auth		
POST	/api/auth/register	Register a new user account.
POST	/api/auth/login	Authenticate user and issue JWT tokens.
POST	/api/auth/logout	Invalidate session and clear authentication cookies.
Patients		
GET	/api/patients	Retrieve a list of all registered patients (Admin).
POST	/api/patients	Create a new patient profile.
GET	/api/patients/:id	Retrieve a specific patient's profile by ID.
PUT	/api/patients/:id	Update a specific patient's profile.
Doctors		
GET	/api/doctors	Retrieve a list of all registered doctors.
POST	/api/doctors	Create a new doctor profile (Admin).

PUT	/api/doctors/:id	Update a specific doctor's profile (Admin).
DELETE	/api/doctors/:id	Remove a doctor's profile from the system (Admin).
Appointments		
GET	/api/appointments	Retrieve appointments (filtered by role).
POST	/api/appointments	Book a new appointment.
PUT	/api/appointments/:id	Update appointment details or status.
DELETE	/api/appointments/:id	Cancel and remove an appointment.
Medical Records		
POST	/api/medical-records	Create a new medical record (Doctor).
GET	/api/medical-records/:patientId	Retrieve all medical records for a specific patient.
Prescriptions		
POST	/api/prescriptions	Issue a new prescription (Doctor).
GET	/api/prescriptions/:patientId	Retrieve all prescriptions for a specific patient.

12. User Interface — Page Structure

The frontend application is organized into role-specific route namespaces. All pages are rendered using Next.js with server-side rendering where appropriate.

Role	Route	Description
Admin	/admin/dashboard	Platform overview with key metrics (patients, doctors, appointments).
Admin	/admin/patients	Patient account management interface.
Admin	/admin/doctors	Doctor account and profile management interface.
Admin	/admin/appointments	System-wide appointment management and oversight.
Doctor	/doctor/dashboard	Physician's personal dashboard with upcoming appointments.
Doctor	/doctor/appointments	Detailed view of all assigned appointments with filtering.
Doctor	/doctor/medical-records	Interface for creating and reviewing medical records.
Doctor	/doctor/prescriptions	Interface for issuing and reviewing prescriptions.
Patient	/patient/dashboard	Patient overview with upcoming appointments and recent activity.
Patient	/patient/appointments	Appointment booking, history, and management interface.
Patient	/patient/records	Read-only view of personal medical records.
Patient	/patient/prescriptions	Read-only view of prescriptions issued by treating doctors.

13. Non-Functional Requirements

Requirement	Specification
Performance	The system shall deliver fast page load times through React Query caching and optimized API design. Appointment booking operations shall handle concurrent requests without degradation.
Security	All passwords shall be hashed using bcrypt. Authentication shall be enforced via JWT with HTTP-only cookie storage. Role-based access control shall restrict all sensitive endpoints. All user inputs shall be validated server-side.
Responsiveness	The application interface shall be fully responsive and operationally compatible across mobile, tablet, and desktop viewport sizes.
Scalability	The system architecture shall be modular and extensible to accommodate future additions including billing, laboratory management, admission tracking, nurse dashboards, and analytics reporting.
Maintainability	Codebase shall be organized into clearly delineated modules: auth, appointments, patients, doctors, medical-records, and prescriptions. Each module shall be independently maintainable.
Reliability	The system shall prevent double-booking through server-side conflict detection. All error conditions shall be handled gracefully with appropriate HTTP status codes and user-facing messages.

14. Planned Future Enhancements

The HMS MVP has been architecturally designed to accommodate the following planned modules in subsequent development phases:

Module	Description
Billing System	Automated generation of patient invoices, payment tracking, and financial reporting.
Laboratory Management	Integration with diagnostic laboratories for test ordering, sample tracking, and result reporting.
Admission Management	Workflow management for inpatient admissions, discharge processing, and ward assignment.
Bed Management	Real-time tracking of ward bed availability, occupancy status, and allocation.
Nurse Dashboard	Dedicated interface for nursing staff to manage patient vitals, care plans, and shift handovers.
Analytics & Reporting	Data visualization and reporting tools for administrative decision-making and performance monitoring.

15. Conclusion

The Hospital Management System MVP represents a production-grade, full-stack healthcare platform engineered to modernize and automate the core operational workflows of a clinical environment. By combining a secure authentication framework, role-based access control, real-time appointment management, structured medical record documentation, and digital prescription issuance, the system delivers immediate operational value while establishing a robust foundation for future growth.

The defined five-day delivery schedule ensures that all MVP components are developed, integrated, and validated within a structured and traceable timeframe. The modular architecture guarantees that the system is prepared for hospital-scale expansion without requiring architectural refactoring.

This document shall serve as the authoritative reference for all development, review, and stakeholder communication activities related to the HMS MVP.